

Project 5: BNN

GITHUB LINK: <https://github.com/JDRadatti/cse237c>

DEMO LINK:  [project5recording.mp4](#)

Testbench Integration

In `bnn_mnist.py` we created our header file `golden.h`. This header file contains the input bits from `mnist-bit_sample10.npy` and our expected output from `mnist_test_data_original.npy`. The header also contains each layer output so we can compare our implementation to the original python version. The testbench `bnn_test.cpp` checks each output of each layer with the expected output of each layer that is in our header file, and the final output with the expected output which is also in the header file. The testbench goes through all 10 samples and checks to see if our implementation passes or fails.

Optimizations

The overall optimizations that we chose were to drastically reduce the latency to as low as possible. We have the dataflow optimization which allows us to run the 3 layers in parallel without overlapping each other. We used dataflow because it enables layer-level pipelining, allowing layer 2 to begin processing as soon as layer 1 produces outputs rather than waiting for the entire batch to complete, significantly reducing end-to-end latency. When layer 1 has finished with some data it can send it to layer 2 without having to wait for layer 1 to fully finish. The trade-offs for this is that it increases area size due to requiring FIFOs between stages.

We partitioned the input bits array so that all 25 bits can be accessed in parallel. We chose complete partitioning for the input array because the 25 input values need to be accessed simultaneously in every neuron computation, and this eliminates all memory access bottlenecks in the critical first layer. The trade-offs for this is that we have to use registers instead of BRAM now.

We did cyclic array partitioning on layers 1 and 2 and complete array partitioning on layer 3. We applied cyclic partitioning to the weight arrays in layers 1 and 2 (with factors of 100 and 32 respectively) because these large arrays (3200 and 256 elements) need multiple parallel accesses per cycle when processing multiple neurons simultaneously, and cyclic partitioning provides the necessary memory bandwidth while still using BRAM rather than exhausting the register file. For layer 3, we used complete partitioning because it's small enough (only 20 elements) that the register cost is negligible and it provides maximum access bandwidth. The trade-offs for cyclic partitioning is that now we have to use more BRAM blocks instead of 1 BRAM block.

We also implemented loop unrolling and pipelining in all of the layers. Loop unrolling was fully applied to the weight accumulation loops (unrolling all 25, 4, and 2 iterations respectively) to allow all weight-input XNOR operations for a single neuron to compute in parallel. We also process multiple neurons per outer loop iteration (4 in layer 1, 2 in layer 2, and 1 in layer 3) to increase parallelism. Loop pipelining with $II=1$ was applied to the outer neuron loops to process these neuron groups in an overlapped fashion, maximizing throughput by starting a new group of neurons every clock cycle. Additionally, the bit-packing loops in layers 2 and 3 use partial unrolling (factor=8) to balance speed and resource usage when converting integer activations to binary format. Trade-offs for unrolling is that it requires more LUTs/DSPs and routing resources scale with the unroll factor. The trade-offs for loop pipelining are that it requires more registers for the pipeline stages.

The outputs of layer 1 and layer 2 get converted into FIFO queues which enables pipelining without the full buffer memory. We used streaming FIFOs because they provide the necessary buffering for dataflow between stages while using far less memory than full ping-pong buffers would require, and the shallow depth keeps the stages tightly coupled for minimum latency. The trade-offs for using FIFOs is that it uses BRAMs or distributed RAM.

Overall the optimizations that we chose were to minimize latency at the expense of increased resource utilization.

HLS IP Core Interface

We created three AXI ports to interface with our BNN using the “#pragma HLS INTERFACE s_axilite port” compiler directive. The main two ports we used were called “input_bits” and “ys”. “input_bits” represents the input to the BNN - we fed test data from an MNIST data set to this port. “ys” represents the output of the BNN. We used Vivado’s automatic routing to connect our BNN IP to our board’s CPU and generated bitstream and hardware handoff files using the default Vivado project options and workflow.

Notebook Demo

Our Jupiter notebook tests our BNN implementation on our FPGA using the same data as our testbench. The notebook first loads the bitstream of our implementation as an overlay and then gets our BNN IP. For each of the test cases, we input the array of test data to the BNN IP. This is achieved by writing the elements of the input array to the range of memory addresses corresponding to the “input_bits” port of the BNN IP. The notebook then sends a start signal to the IP to start its execution. It blocks until it receives a return signal from the IP. Lastly, the notebook reads outputs of the BNN and prints them.

